

RDFIA · LECTURE 2

Neural networks & ML theory

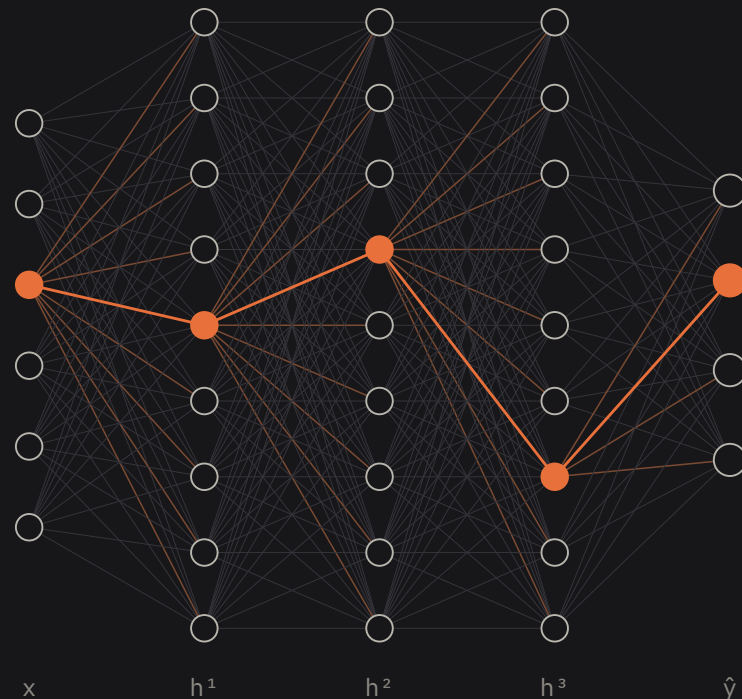
Formal neurons, training by backpropagation,
and statistical decision theory

Clement Rambour

Sorbonne University

clement.rambour@sorbonne-universite.fr

clementrambour.github.io/teaching-rdfia



Course outline

01 Intro to computer vision and machine learning

02 Intro to neural networks + ML theory Today

03 Neural nets for image classification

04 Large ConvNets

05 Vision transformers

06 Segmentation, transfer learning, domain adaptation

07 Vision-language models

08 Explaining VLMs

09 Self-supervised learning in vision

10 Generative models with GANs

11 **Control** Jan 07, 2027

12 Diffusion models

13 Bayesian deep learning

14 Uncertainty, robustness

PART 1 / 3

Introduction to neural nets

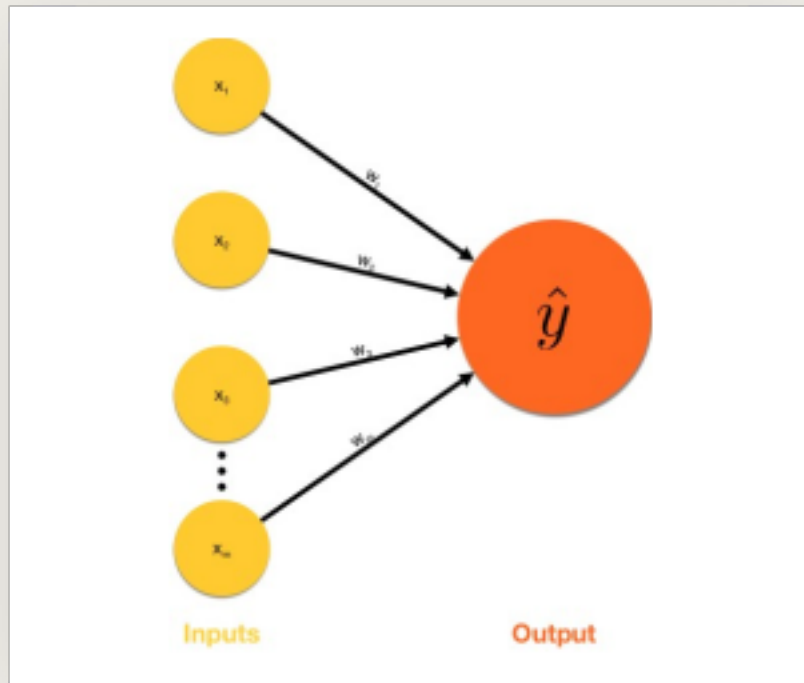
01 Introduction to neural nets

02 Training deep neural networks

03 Introduction to statistical decision theory

The formal neuron

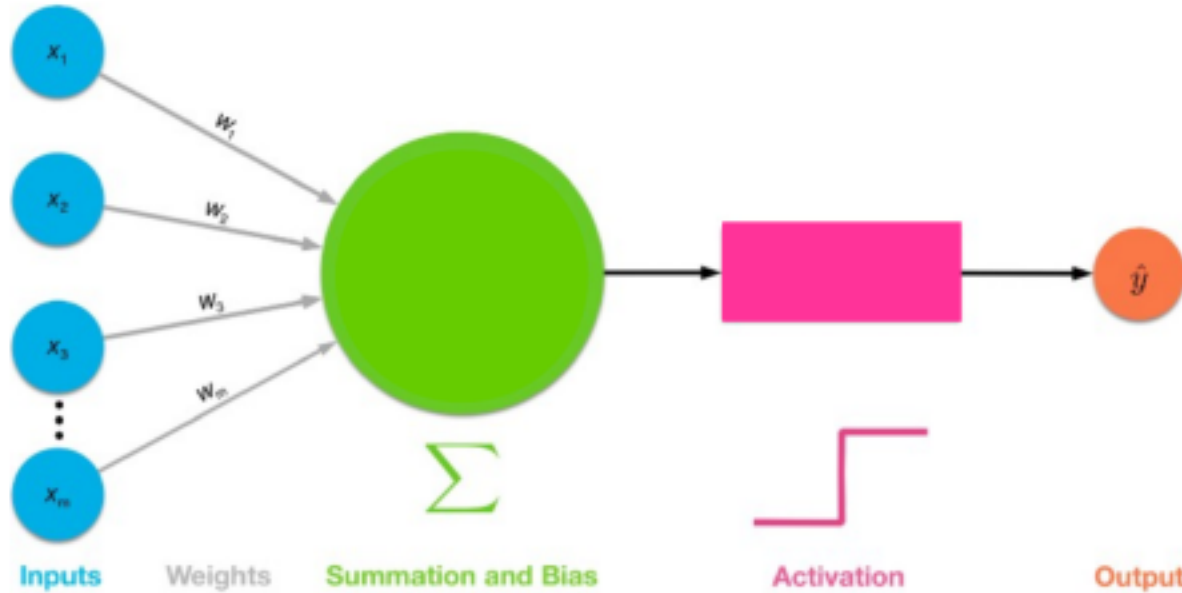
- Basis of neural networks
- Input: vector $\mathbf{x} \in \mathbb{R}^m$, i.e. $\mathbf{x} = \{x_i\}_{i \in \{1, \dots, m\}}$
- Neuron output $\hat{y} \in \mathbb{R}$: scalar



From $\mathbf{x}$ to $\hat{y}$: two steps

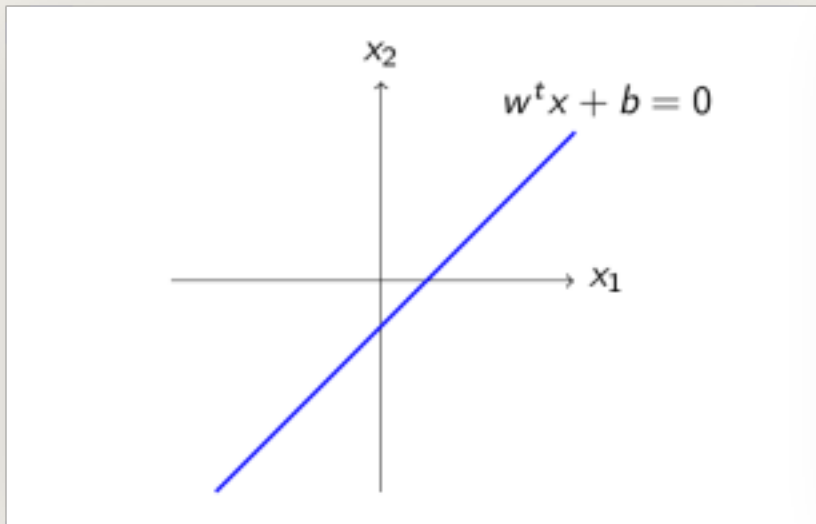
01 Linear (affine) mapping: $s = \mathbf{w}^\top \mathbf{x} + b$

02 Non-linear activation $f: \hat{y} = f(s)$

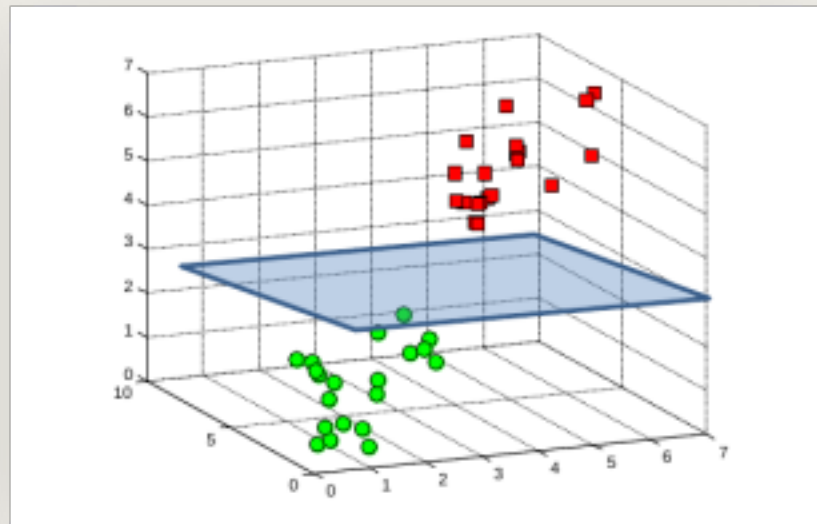


Linear mapping

- Linear (affine) mapping: $s = \mathbf{w}^\top \mathbf{x} + b = \sum_{i=1}^m w_i x_i + b$
 - $\mathbf{w}$: normal vector to a hyperplane in $\mathbb{R}^m \Rightarrow$ **linear boundary**
 - b : bias, shifts the hyperplane position



2D hyperplane: line



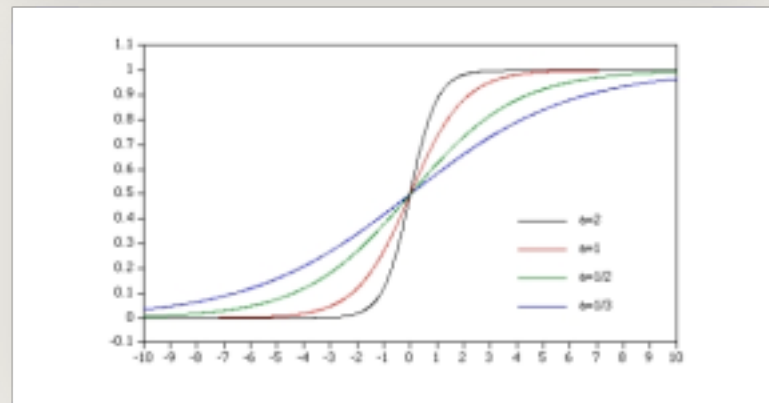
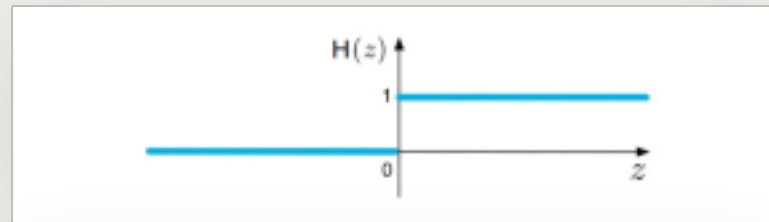
3D hyperplane: plane

Activation function

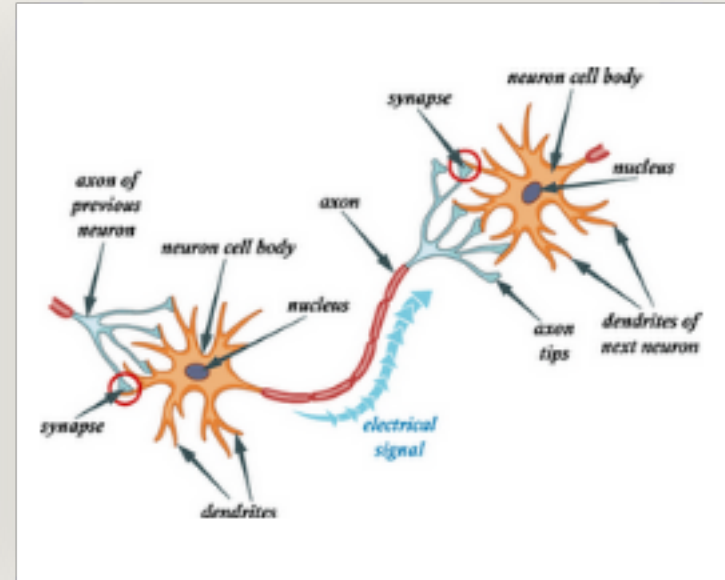
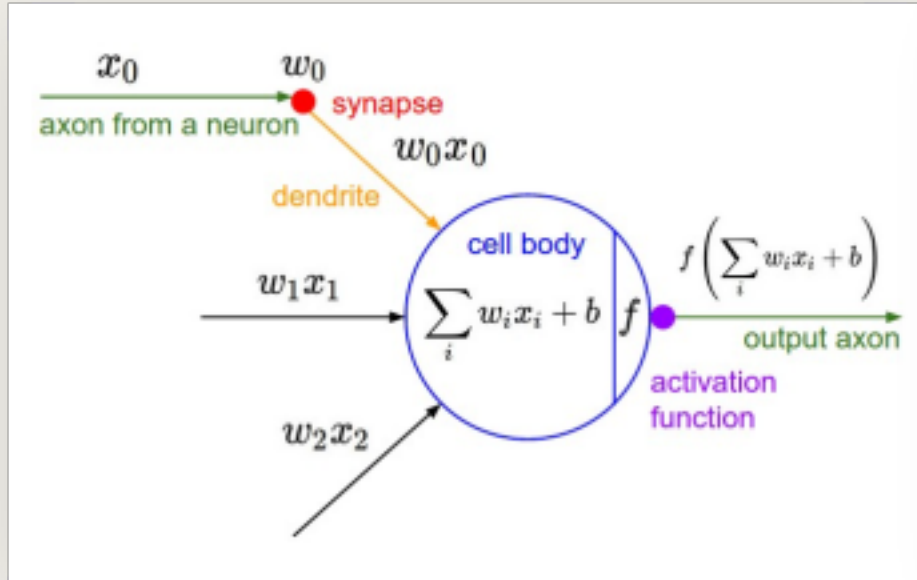
- $\hat{y} = f(\mathbf{w}^\top \mathbf{x} + b)$, f : activation function
- Bio-inspired choice — step (Heaviside) function:

$$H(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

- Popular choices: sigmoid, tanh, ReLU, GELU, ...
- Sigmoid: $\sigma(z) = (1 + e^{-az})^{-1}$
 - $a \uparrow$: closer to the step function (step: $a \rightarrow \infty$)
 - Linear and saturating regimes



Step function: connection to biological neurons

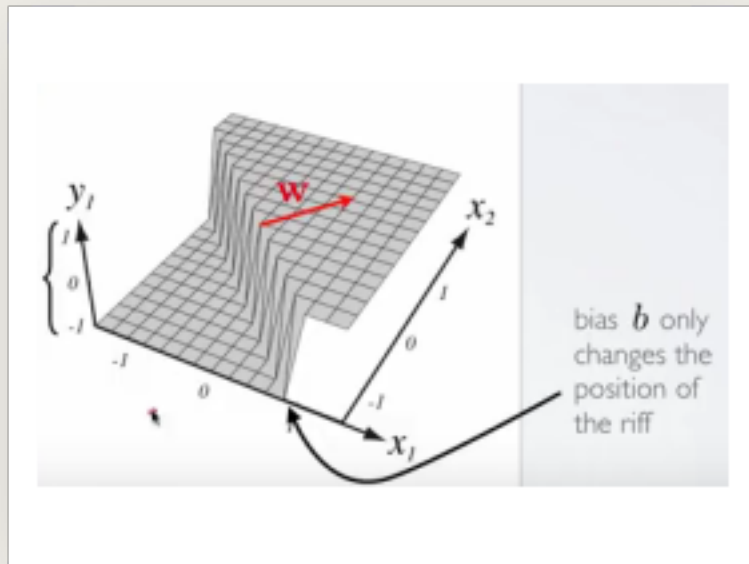


- Formal neuron, step activation: $\hat{y} = H(\mathbf{w}^\top \mathbf{x} + b)$
 - $\hat{y} = 1$ (activated) $\Leftrightarrow \mathbf{w}^\top \mathbf{x} \geq -b$
 - $\hat{y} = 0$ (unactivated) $\Leftrightarrow \mathbf{w}^\top \mathbf{x} < -b$

- Biological neurons: output activated $\Leftrightarrow$ input weighted by synaptic weight $\geq$ threshold

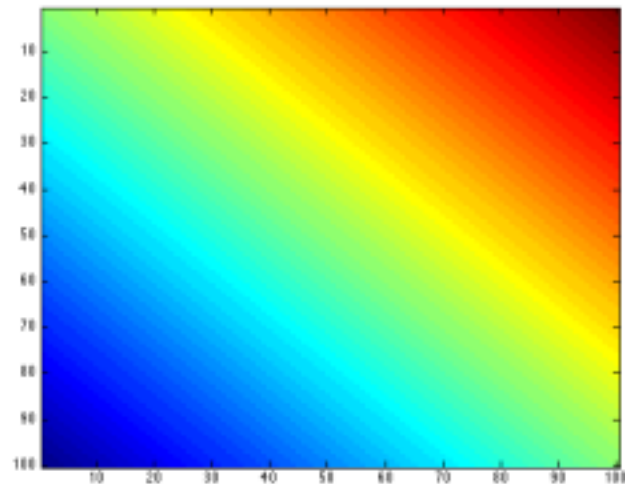
Application to binary classification

- **Binary classification:** label input $\mathbf{x}$ as class 1 or 0
- Neuron output with sigmoid: $\hat{y} = \frac{1}{1 + e^{-a(\mathbf{w}^\top \mathbf{x} + b)}}$
- Probabilistic interpretation $\Rightarrow \hat{y} \sim P(1 | \mathbf{x})$
 - Class 1 if $P(1 | \mathbf{x}) > 0.5 \Leftrightarrow \mathbf{w}^\top \mathbf{x} + b > 0$
 - Class 0 if $P(1 | \mathbf{x}) < 0.5 \Leftrightarrow \mathbf{w}^\top \mathbf{x} + b < 0$
- $\Rightarrow \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$: **linear decision boundary** in input space



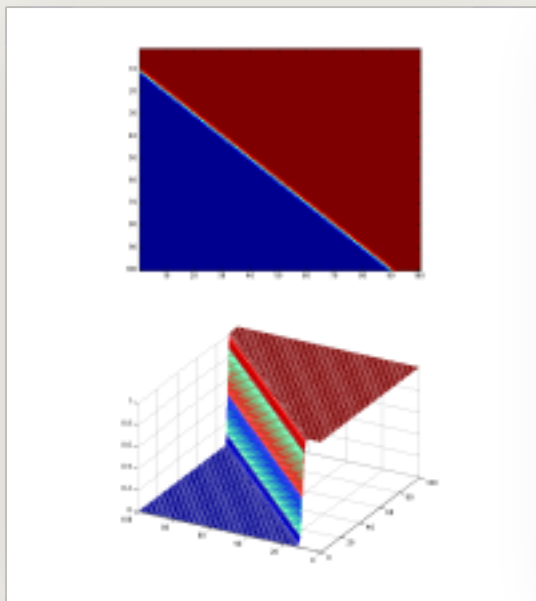
Result of the linear mapping

- 2D example: $m = 2$, $\mathbf{x} = \{x_1, x_2\} \in [-5; 5] \times [-5; 5]$
- Linear mapping: $\mathbf{w} = [1; 1]$ and $b = -2$
- Result: $s = \mathbf{w}^\top \mathbf{x} + b$

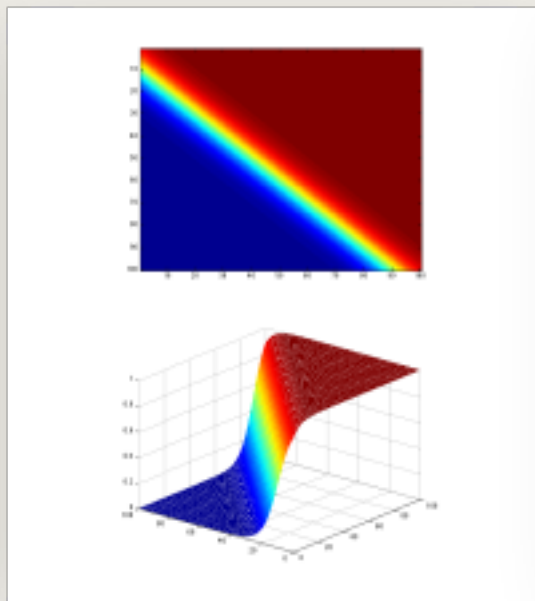


Sigmoid activation: effect of a

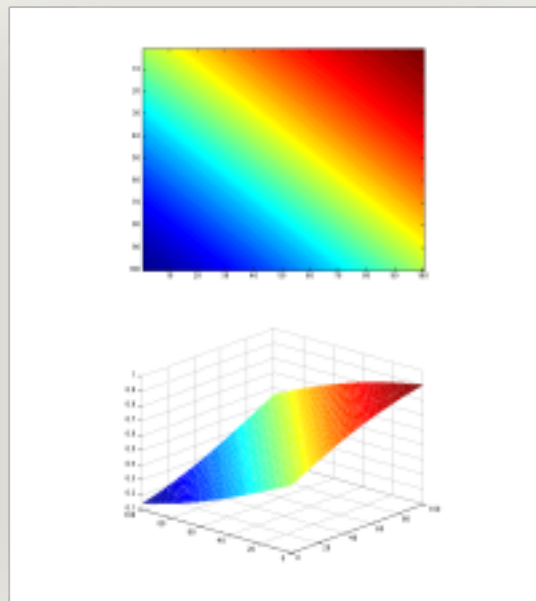
$$\hat{y} = \left(1 + e^{-a(\mathbf{w}^\top \mathbf{x} + b)}\right)^{-1}, \text{ with } \mathbf{w} = [1; 1], b = -2$$



$a = 10$



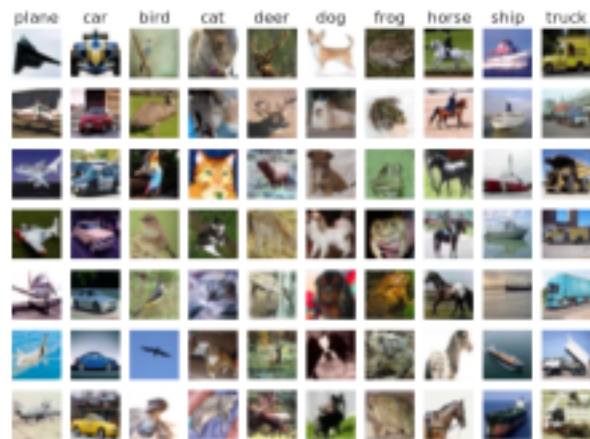
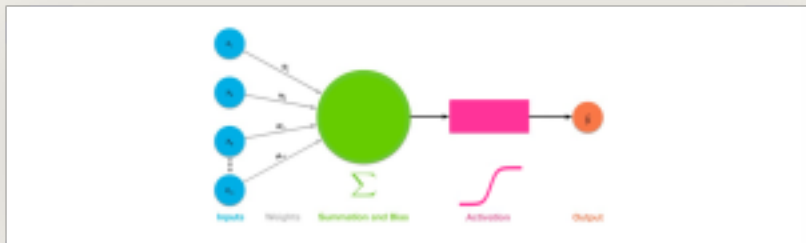
$a = 1$



$a = 0.1$

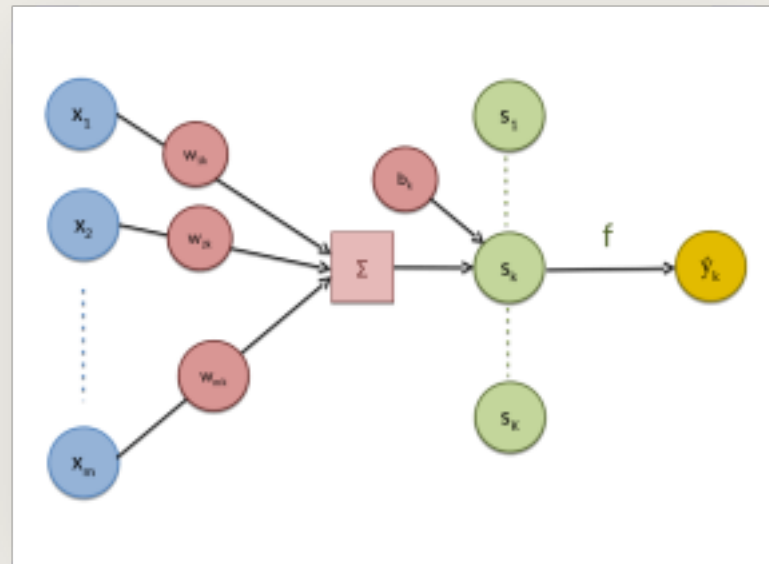
From formal neuron to neural networks

- Formal neuron:
 - A single scalar output
 - Linear decision boundary for binary classification
- Single scalar output: **limited for several tasks**
 - e.g. multi-class classification: MNIST or CIFAR



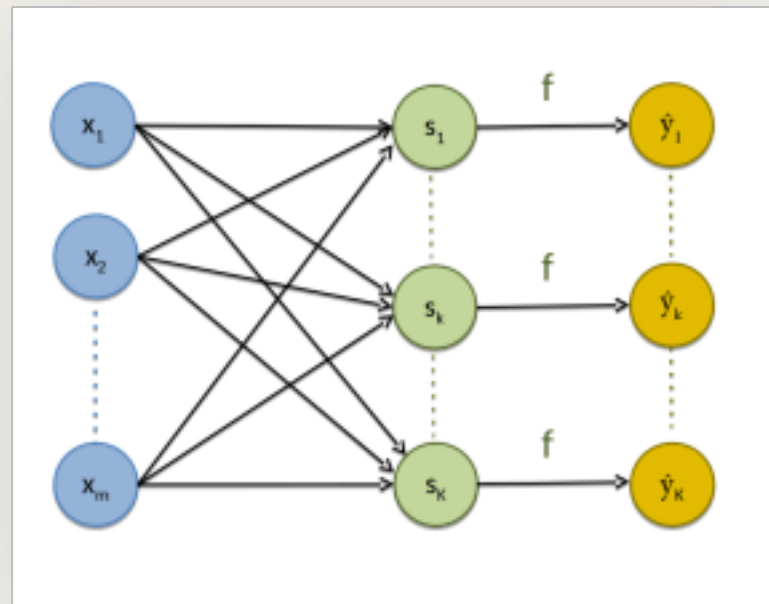
The perceptron

- Formal neuron: limited to binary classification
- **Multi-class:** use several output neurons instead of one
⇒ **Perceptron**
- Input $\mathbf{x} \in \mathbb{R}^m$; output neuron $\hat{y}_k$ is a formal neuron:
 - Linear (affine) mapping: $s_k = \mathbf{w}_k^\top \mathbf{x} + b_k$
 - Non-linear activation: $\hat{y}_k = f(s_k)$
 - Parameters: $\mathbf{w}_k = \{w_{1k}, \dots, w_{mk}\} \in \mathbb{R}^m, b_k \in \mathbb{R}$



Perceptron as a matrix product

- Input $\mathbf{x} \in \mathbb{R}^m$ ($1 \times m$), output $\hat{\mathbf{y}}$: concatenation of K formal neurons
- Linear (affine) mapping $\sim$ matrix multiplication: $\mathbf{s} = \mathbf{x}\mathbf{W} + \mathbf{b}$
 - $\mathbf{W}$: matrix of size $m \times K$ — columns are $\mathbf{w}_k$
 - $\mathbf{b}$: bias vector of size $1 \times K$
- Element-wise non-linear activation: $\hat{\mathbf{y}} = f(\mathbf{s})$

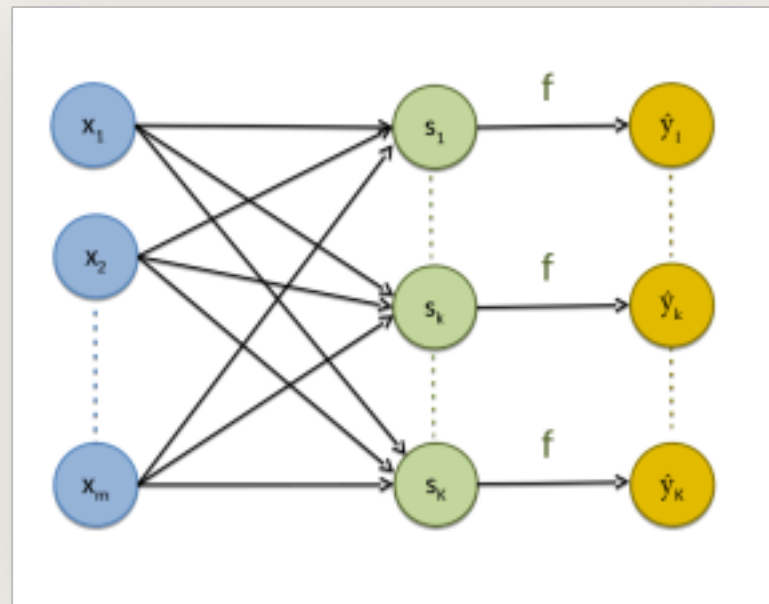


Soft-max activation

$$\hat{y}_k = f(s_k) = \frac{e^{s_k}}{\sum_{k'=1}^K e^{s_{k'}}}$$

- $f(s_k)$ depends on the other $s_{k'}$: the arrow is a functional link
- **Probabilistic interpretation** for multi-class classification:
 - Each output neuron $\Leftrightarrow$ class
 - $\hat{y}_k \sim P(k \mid \mathbf{x}, \mathbf{W})$

⇒ **Logistic Regression (LR) model**

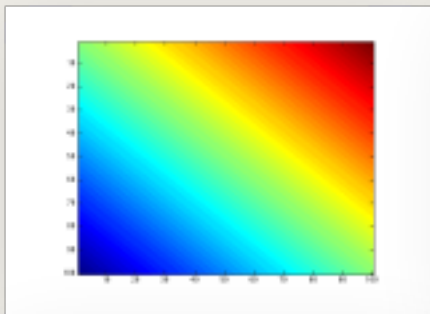


2D multi-class classification

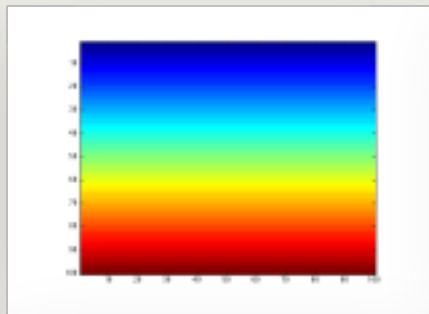
$\mathbf{x} = \{x_1, x_2\} \in [-5; 5] \times [-5; 5]$, $\hat{\mathbf{y}}$: 3 outputs (classes)

Linear mapping per
class: $s_k = \mathbf{w}_k^\top \mathbf{x} + b_k$

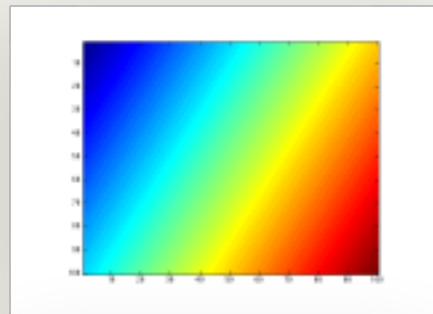
$$\mathbf{w}_1 = [1; 1], \quad b_1 = -2$$



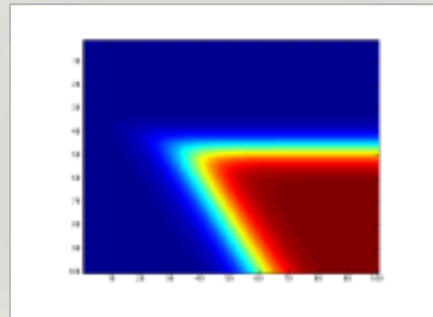
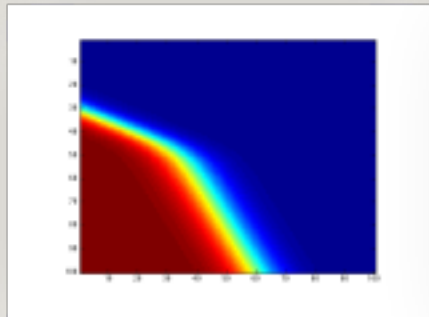
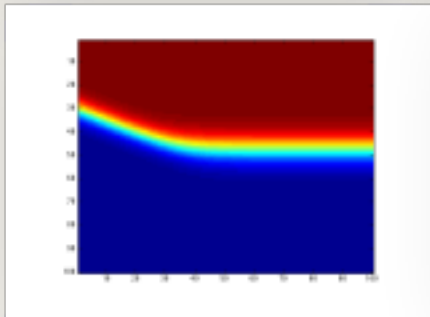
$$\mathbf{w}_2 = [0; -1], \quad b_2 = 1$$



$$\mathbf{w}_3 = [1; -0.5], \quad b_3 = 10$$

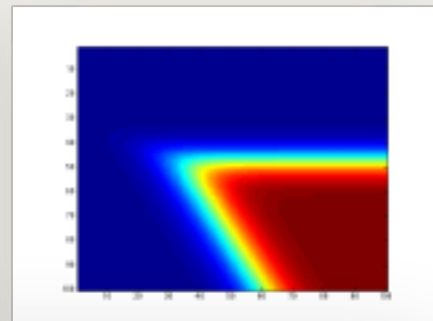
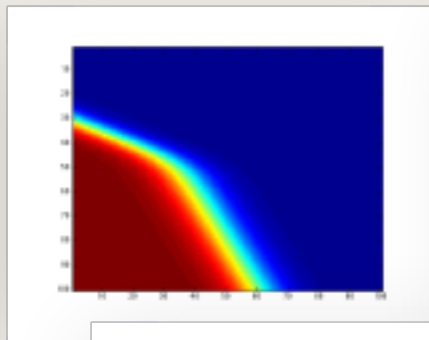
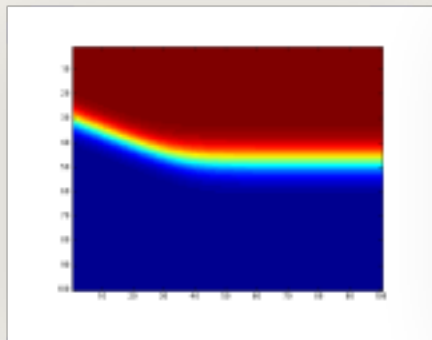


Soft-max output: $P(k | \mathbf{x}, \mathbf{W})$



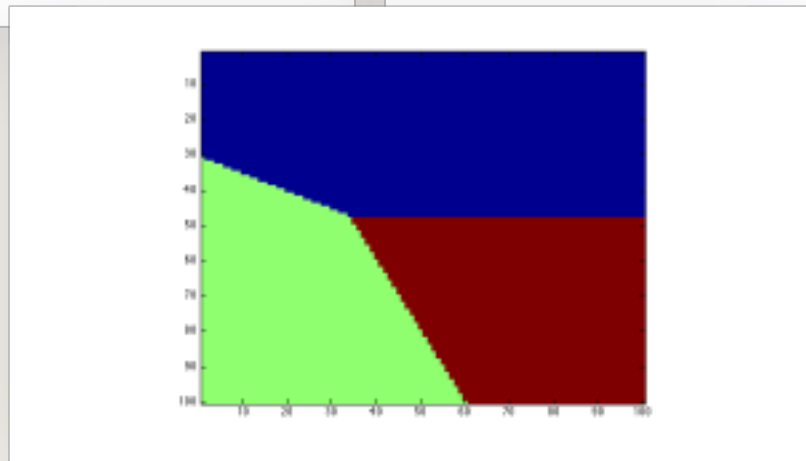
Class prediction

Soft-max output: $P(k \mid \mathbf{x}, \mathbf{W})$



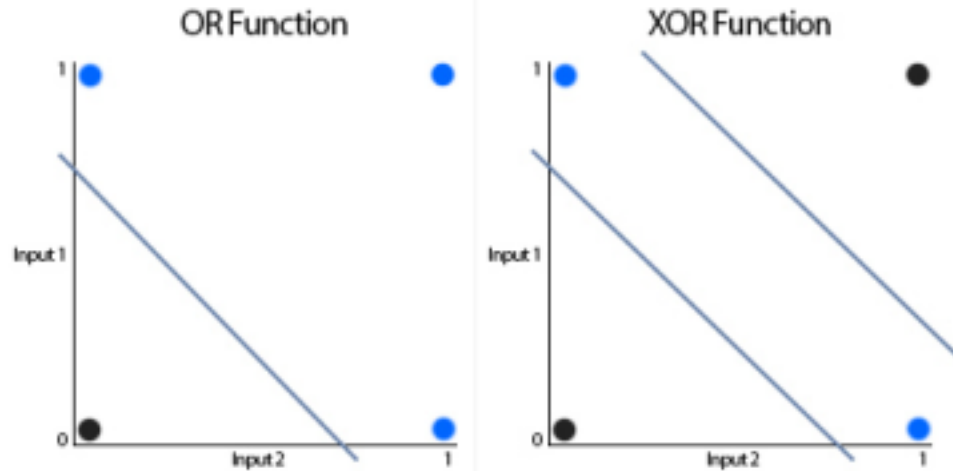
CLASS PREDICTION

$$k^* = \arg \max_k P(k \mid \mathbf{x}, \mathbf{W})$$



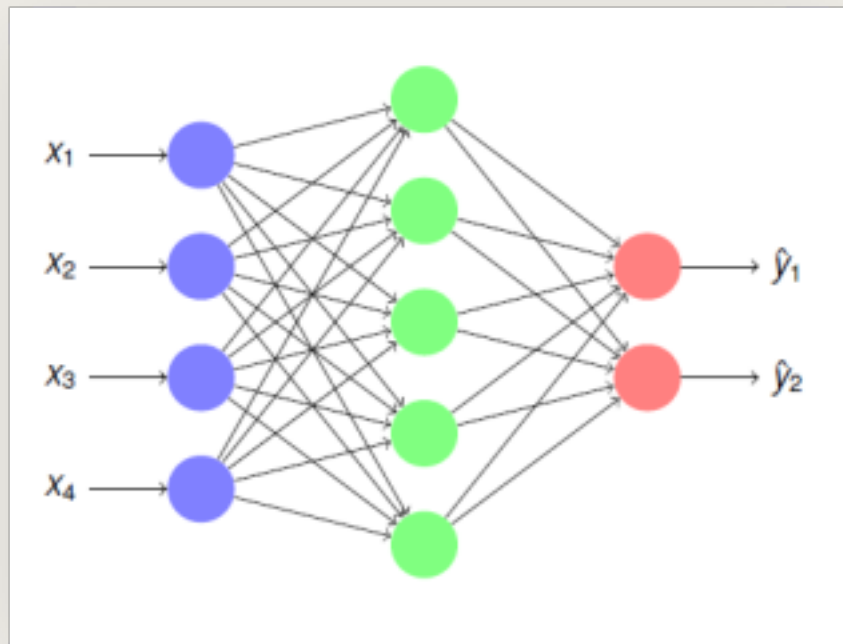
The X-OR problem

- Logistic Regression (LR): NN with 1 input layer & 1 output layer
- LR: limited to linear decision boundaries
- X-OR: (NOT 1 AND 2) OR (NOT 2 AND 1) ⇒ **non-linear decision function**



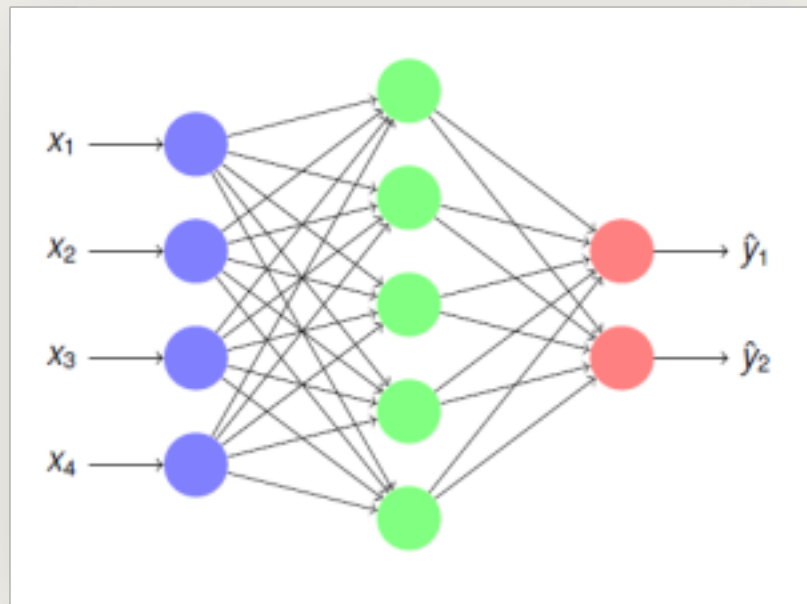
Solution: add a layer

- LR: limited to linear boundaries
- Input $\mathbf{x} \in \mathbb{R}^m$, e.g. $m = 4$
- Output $\hat{\mathbf{y}} \in \mathbb{R}^K$ (K classes), e.g. $K = 2$
- **Hidden layer** $\mathbf{h} \in \mathbb{R}^L$



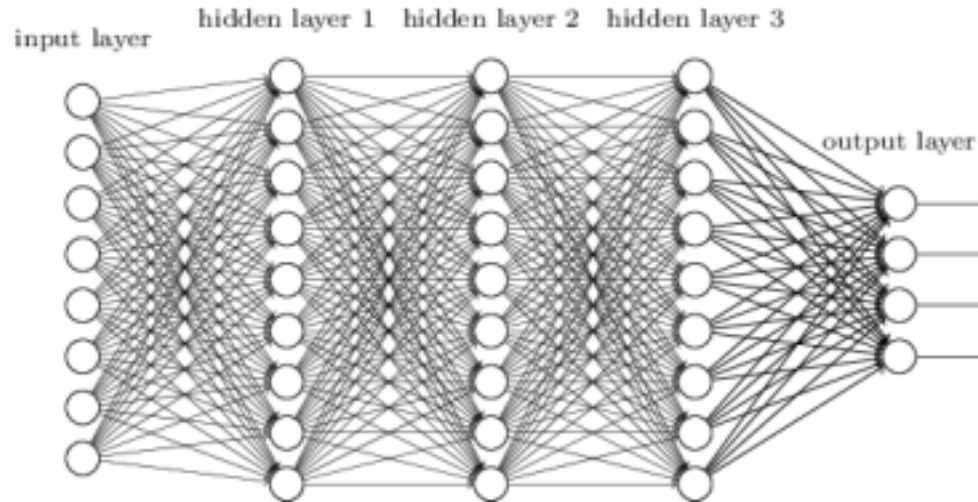
Multi-Layer Perceptron (MLP)

- **Hidden layer $\mathbf{h}$:** projection of $\mathbf{x}$ into a new space $\mathbb{R}^L$
- Neural net with ≥ 1 hidden layer: **Multi-Layer Perceptron**
- $\mathbf{h}$: intermediate representation of $\mathbf{x}$ for classification
- $\mathbf{h} = f(\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)$, f non-linear activation
- $\mathbf{s} = \mathbf{h}\mathbf{W}_2 + \mathbf{b}_2$, $\hat{\mathbf{y}} = \text{SoftMax}(\mathbf{s})$
- Mapping $\mathbf{x} \rightarrow \hat{\mathbf{y}}$: non-linear boundary $\Rightarrow$ **non-linear f is crucial**



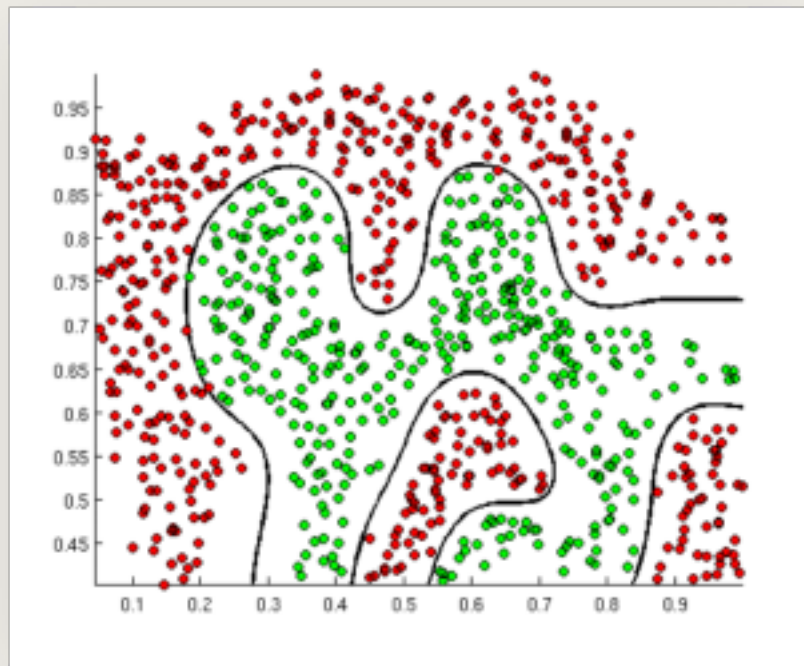
Deep neural networks

- Adding more hidden layers: **Deep Neural Networks (DNN)** $\Rightarrow$ basis of deep learning
- Each layer $\mathbf{h}^l$ projects layer $\mathbf{h}^{l-1}$ into a new space
- Gradually learning intermediate representations useful for the task



Conclusion

- Deep neural networks: applicable to classification problems with **non-linear decision boundaries**
- So far: visualize predictions from *fixed* model parameters
- Reverse problem: **supervised learning**



PART 2 / 3

Training deep neural networks

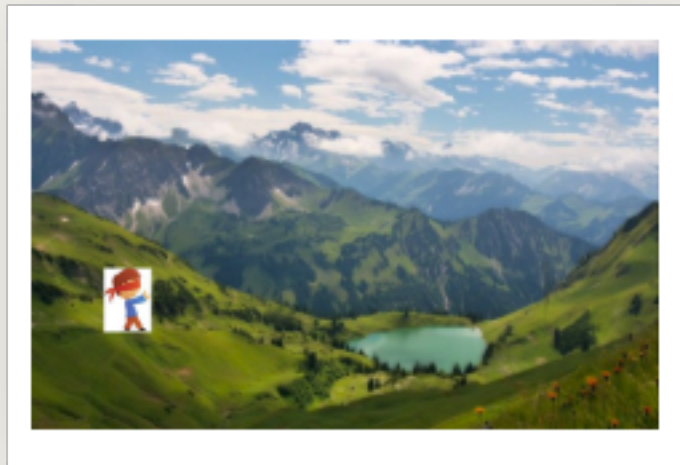
01 Introduction to neural nets

02 Training deep neural networks

03 Introduction to statistical decision theory

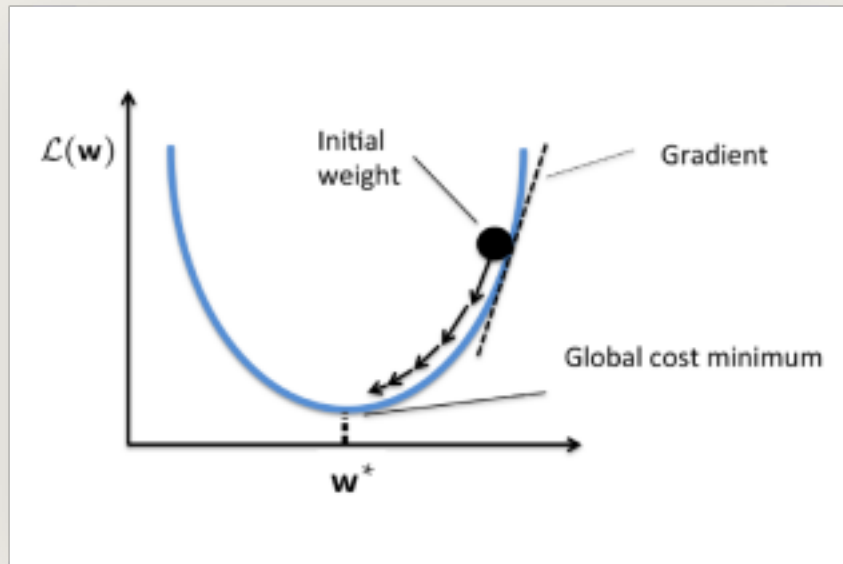
Training a Multi-Layer Perceptron

- A parametrized ($\mathbf{w}$) model $\mathbf{x} \Rightarrow y: f_{\mathbf{w}}(\mathbf{x}_i) = \hat{y}_i$
- Supervised context:
 - Training set $\mathcal{A} = \{(\mathbf{x}_i, y_i^*)\}_{i \in \{1, \dots, N\}}$
 - Loss $\ell(\hat{y}_i, y_i^*)$ for each annotated pair
 - Goal: minimize the average loss $\mathcal{L}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \ell(\hat{y}_i, y_i^*)$
- Assumptions: $\mathbf{w} \in \mathbb{R}^d$ continuous, $\mathcal{L}$ differentiable
- Gradient $\nabla_{\mathbf{w}} = \frac{\partial \mathcal{L}}{\partial \mathbf{w}}$: steepest direction to decrease $\mathcal{L}(\mathbf{w})$



MLP training: gradient descent

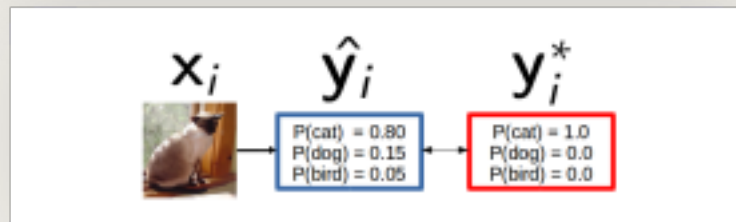
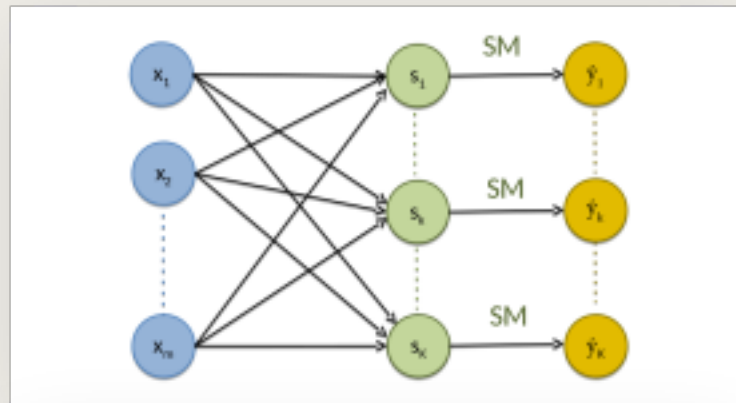
- 01 Initialize parameters $\mathbf{w}$
- 02 Update: $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{w}}$
- 03 Until convergence, e.g. $\|\nabla_{\mathbf{w}}\|^2 \approx 0$



Multi-class classification

- Logistic regression for multi-class classification
- $\mathbf{s}_i = \mathbf{x}_i \mathbf{W} + \mathbf{b}$
- Soft-max: $\hat{y}_k \sim P(k \mid \mathbf{x}_i, \mathbf{W}, \mathbf{b}) = \frac{e^{s_k}}{\sum_{k'=1}^K e^{s_{k'}}}$
- Supervised loss: $\mathcal{L}(\mathbf{W}, \mathbf{b}) = \frac{1}{N} \sum_{i=1}^N \ell(\hat{\mathbf{y}}_i, \mathbf{y}_i^*)$
- Input $\mathbf{x}_i$, **ground-truth supervision** $\mathbf{y}_i^*$, one-hot encoded:

$$y_{c,i}^* = \begin{cases} 1 & \text{if } c \text{ is the ground-truth class of } \mathbf{x}_i \\ 0 & \text{otherwise} \end{cases}$$

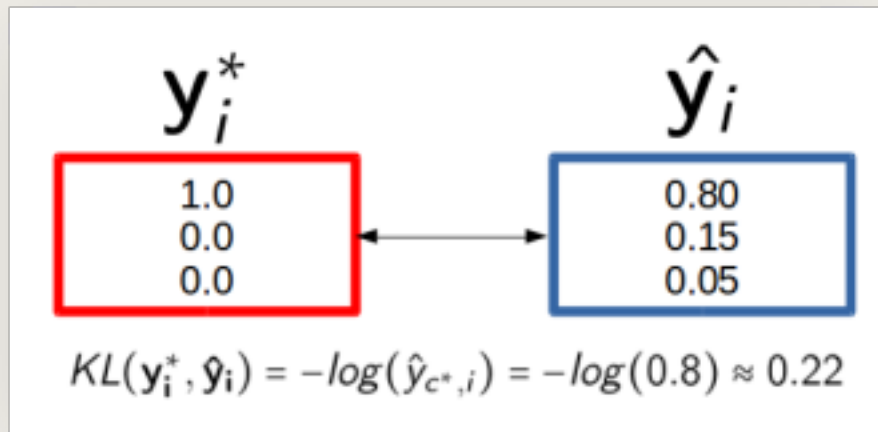


Training formulation: cross-entropy

- Loss function: multi-class Cross-Entropy (CE) ℓ_{CE}
- ℓ_{CE} : Kullback-Leibler divergence between $\mathbf{y}_i^*$ and $\hat{\mathbf{y}}_i$

$$\ell_{CE}(\hat{\mathbf{y}}_i, \mathbf{y}_i^*) = KL(\mathbf{y}_i^*, \hat{\mathbf{y}}_i) = - \sum_{c=1}^K y_{c,i}^* \log(\hat{y}_{c,i}) = - \log(\hat{y}_{c^*,i})$$

NOTE KL is asymmetric: $KL(\hat{\mathbf{y}}_i, \mathbf{y}_i^*) \neq KL(\mathbf{y}_i^*, \hat{\mathbf{y}}_i)$



Training by gradient descent

- $\mathcal{L}_{CE}(\mathbf{W}, \mathbf{b}) = \frac{1}{N} \sum_{i=1}^N \ell_{CE}(\hat{\mathbf{y}}_i, \mathbf{y}_i^*) = -\frac{1}{N} \sum_{i=1}^N \log(\hat{y}_{c^*,i})$
- ℓ_{CE} : smooth convex upper bound of $\ell_{0/1} \Rightarrow$ **gradient descent optimization**
- Gradient descent: $\mathbf{W}^{(t+1)} = \mathbf{W}^{(t)} - \eta \frac{\partial \mathcal{L}_{CE}}{\partial \mathbf{W}}$ ($\mathbf{b}^{(t+1)} = \mathbf{b}^{(t)} - \eta \frac{\partial \mathcal{L}_{CE}}{\partial \mathbf{b}}$)
- **Main challenge:** computing $\frac{\partial \mathcal{L}_{CE}}{\partial \mathbf{W}} = \frac{1}{N} \sum_{i=1}^N \frac{\partial \ell_{CE}}{\partial \mathbf{W}} ?$

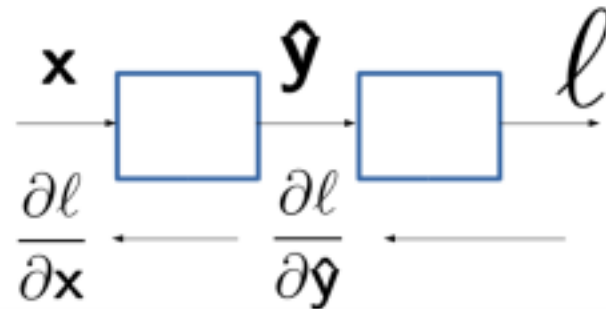
KEY PROPERTY · CHAIN RULE

$$\frac{\partial x}{\partial z} = \frac{\partial x}{\partial y} \frac{\partial y}{\partial z}$$

$\Rightarrow$ Backpropagation of the gradient error

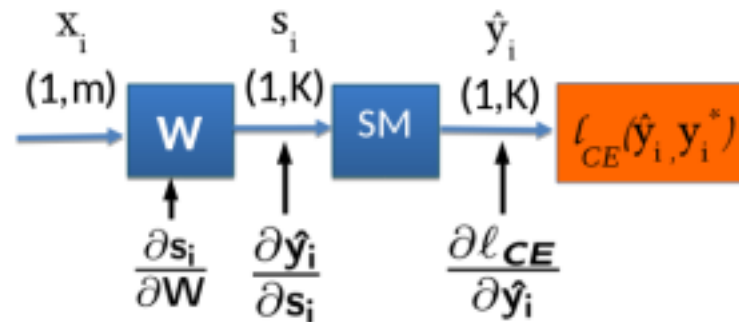
Chain rule

$$\frac{\partial \ell}{\partial x} = \frac{\partial \ell}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial x}$$



Logistic regression:

$$\frac{\partial \ell_{CE}}{\partial \mathbf{W}} = \frac{\partial \ell_{CE}}{\partial \hat{\mathbf{y}}_i} \frac{\partial \hat{\mathbf{y}}_i}{\partial \mathbf{s}_i} \frac{\partial \mathbf{s}_i}{\partial \mathbf{W}}$$



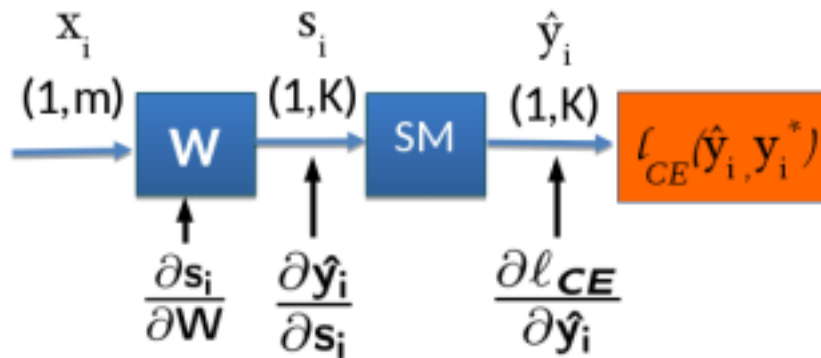
Update for one example

$$\frac{\partial \ell_{CE}}{\partial \mathbf{W}} = \frac{\partial \ell_{CE}}{\partial \hat{\mathbf{y}}_i} \frac{\partial \hat{\mathbf{y}}_i}{\partial \mathbf{s}_i} \frac{\partial \mathbf{s}_i}{\partial \mathbf{W}}, \quad \ell_{CE}(\hat{\mathbf{y}}_i, \mathbf{y}_i^*) = -\log(\hat{y}_{c^*,i})$$

$$\frac{\partial \ell_{CE}}{\partial \hat{\mathbf{y}}_i} = \frac{-1}{\hat{y}_{c^*,i}} = \frac{-1}{\hat{\mathbf{y}}_i} \odot \boldsymbol{\delta}_{c,c^*}$$

$$\frac{\partial \ell_{CE}}{\partial \mathbf{s}_i} = \hat{\mathbf{y}}_i - \mathbf{y}_i^* = \boldsymbol{\delta}_i^y$$

$$\frac{\partial \ell_{CE}}{\partial \mathbf{W}} = \mathbf{x}_i^\top \boldsymbol{\delta}_i^y$$

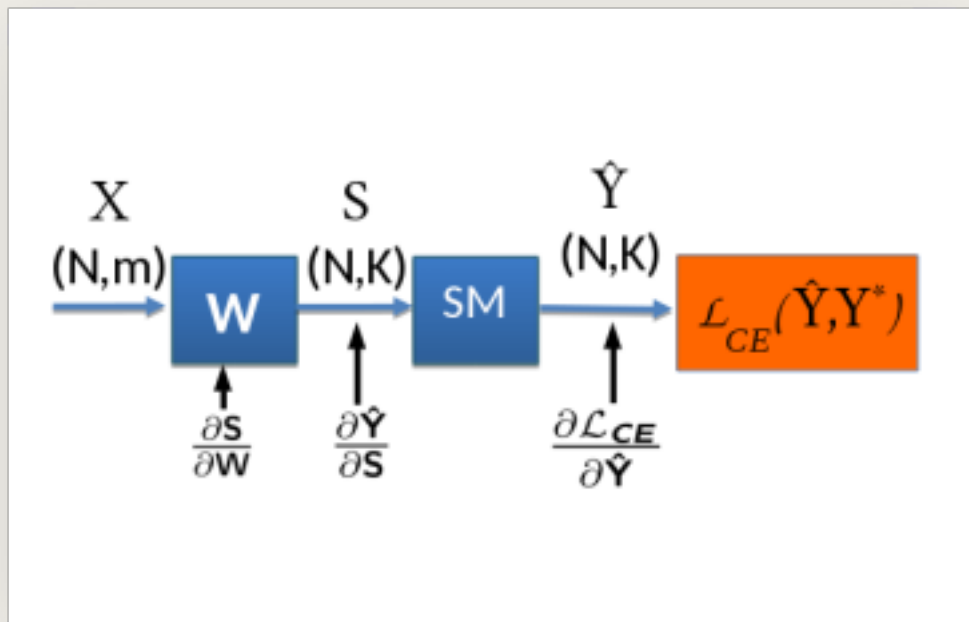


Whole dataset, in matrix form

- Data matrix $\mathbf{X}$ ($N \times m$), label matrices $\hat{\mathbf{Y}}, \mathbf{Y}^*$ ($N \times K$)
- $\mathcal{L}_{CE}(\mathbf{W}, \mathbf{b}) = -\frac{1}{N} \sum_{i=1}^N \log(\hat{y}_{c^*,i}), \quad \frac{\partial \mathcal{L}_{CE}}{\partial \mathbf{W}} = \frac{\partial \mathcal{L}_{CE}}{\partial \hat{\mathbf{Y}}} \frac{\partial \hat{\mathbf{Y}}}{\partial \mathbf{S}} \frac{\partial \mathbf{S}}{\partial \mathbf{W}}$

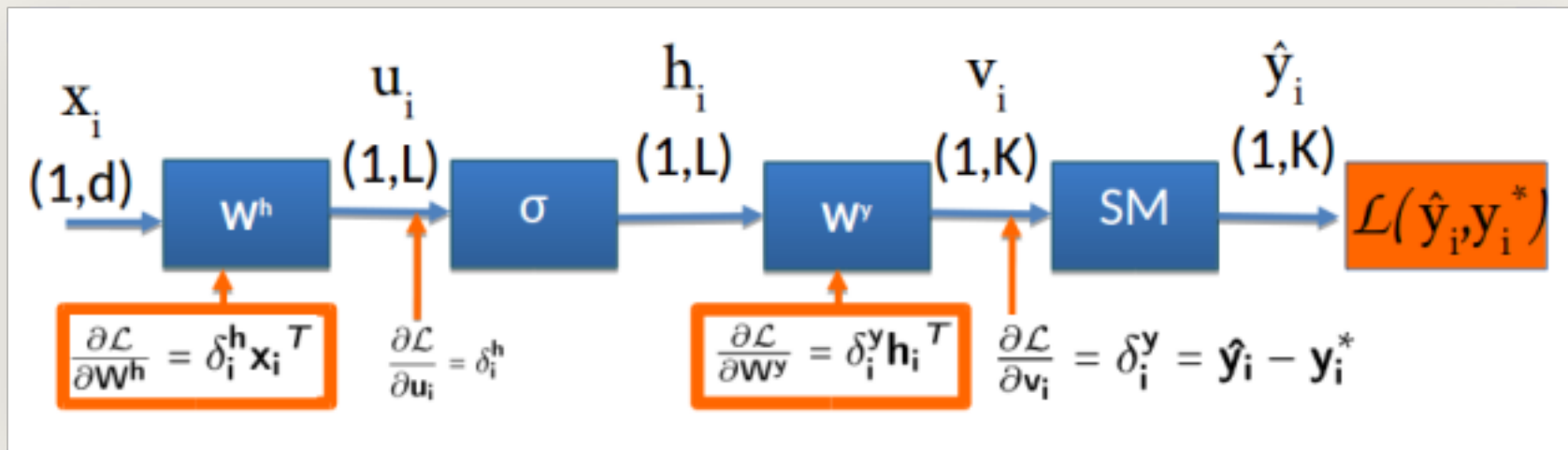
$$\frac{\partial \mathcal{L}_{CE}}{\partial \mathbf{S}} = \hat{\mathbf{Y}} - \mathbf{Y}^* = \Delta^y$$

$$\frac{\partial \mathcal{L}_{CE}}{\partial \mathbf{W}} = \mathbf{X}^\top \Delta^y$$



Adding a hidden layer

- Perceptron vs logistic regression: adding a hidden layer (sigmoid)
- Goal:** train $\mathbf{W}^y$ and $\mathbf{W}^h$ (+ bias) with backpropagation $\Rightarrow$ compute $\frac{\partial \mathcal{L}_{CE}}{\partial \mathbf{W}^y} = \frac{1}{N} \sum_i \frac{\partial \ell_{CE}}{\partial \mathbf{W}^y}$ and $\frac{\partial \mathcal{L}_{CE}}{\partial \mathbf{W}^h} = \frac{1}{N} \sum_i \frac{\partial \ell_{CE}}{\partial \mathbf{W}^h}$

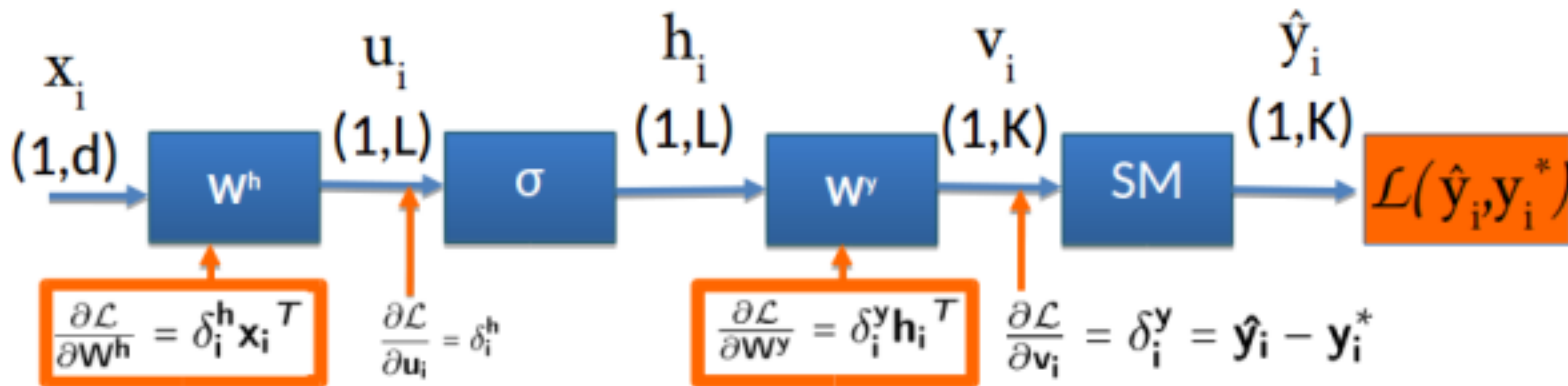


- Last hidden layer $\sim$ logistic regression
- First hidden layer: $\frac{\partial \ell_{CE}}{\partial \mathbf{W}^h} = \mathbf{x}_i^T \frac{\partial \ell_{CE}}{\partial \mathbf{u}_i} \Rightarrow$ **compute** $\frac{\partial \ell_{CE}}{\partial \mathbf{u}_i} = \delta_i^h$

Gradient w.r.t. the hidden layer

- Computing $\frac{\partial \ell_{CE}}{\partial \mathbf{u}_i} = \delta_i^h \Rightarrow$ use the chain rule: $\frac{\partial \ell_{CE}}{\partial \mathbf{u}_i} = \frac{\partial \ell_{CE}}{\partial \mathbf{v}_i} \frac{\partial \mathbf{v}_i}{\partial \mathbf{h}_i} \frac{\partial \mathbf{h}_i}{\partial \mathbf{u}_i}$
- ... leading to:

$$\frac{\partial \ell_{CE}}{\partial \mathbf{u}_i} = \delta_i^h = \delta_i^y \mathbf{W}^{y\top} \odot \sigma'(\mathbf{h}_i) = \delta_i^y \mathbf{W}^{y\top} \odot (\mathbf{h}_i \odot (1 - \mathbf{h}_i))$$



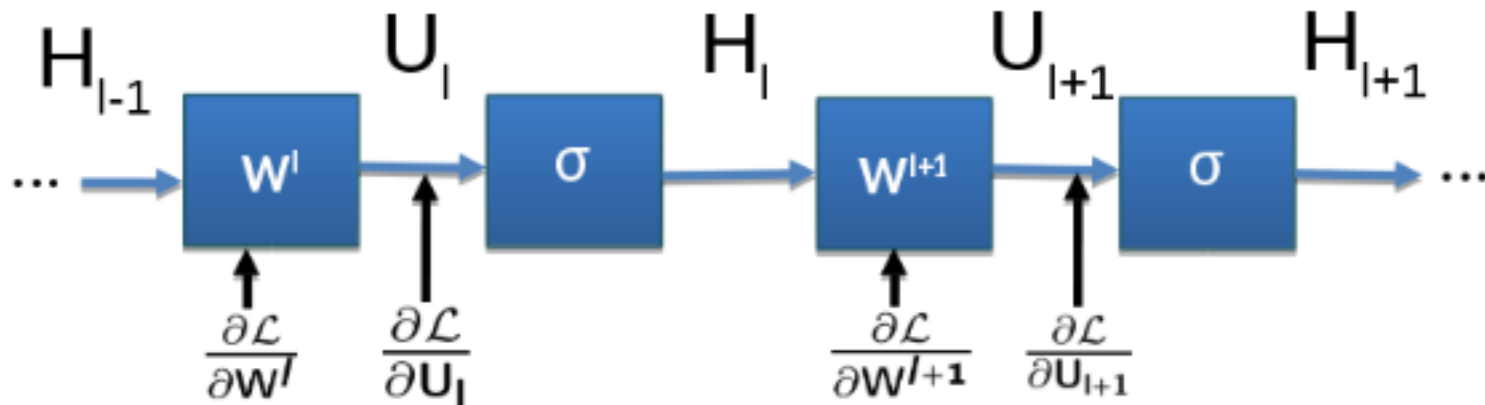
Deep neural network training

- Multi-Layer Perceptron: adding more hidden layers
- Backpropagation update ~ perceptron: **assuming** $\frac{\partial \mathcal{L}}{\partial \mathbf{U}^{l+1}} = \Delta^{l+1}$ **known**

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{l+1}} = \mathbf{H}^l \top \Delta^{l+1}$$

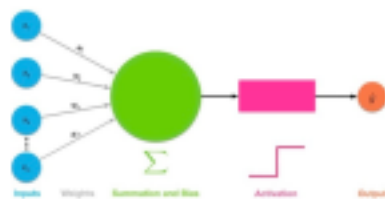
$$\frac{\partial \mathcal{L}}{\partial \mathbf{U}^l} = \Delta^l = \Delta^{l+1} \mathbf{W}^{l+1 \top} \odot \mathbf{H}^l \odot (1 - \mathbf{H}^l)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^l} = \mathbf{H}^{l-1 \top} \Delta^l$$

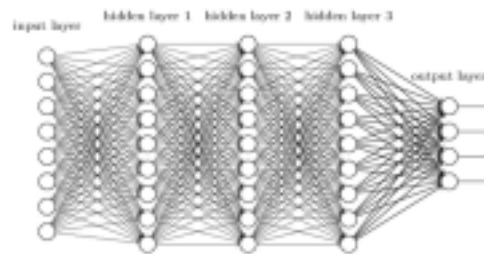
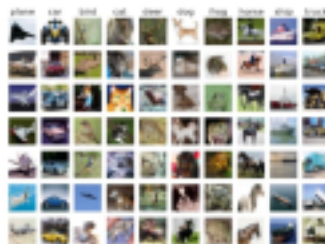


PART 2 · RECAP

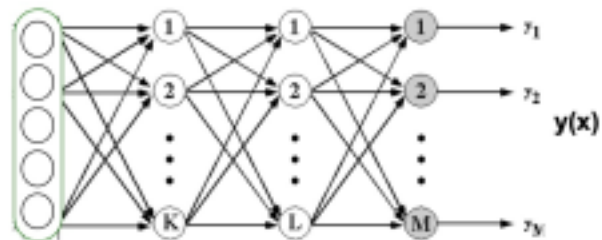
Recap: MLP



Recap MLP



$\longrightarrow$ $\mathbf{x}$



PART 3 / 3

Introduction to statistical decision theory

01 Introduction to neural nets

02 Training deep neural networks

03 Introduction to statistical decision theory

Setting

- $X \in \mathbb{R}^p$: real-valued random input vector
- $Y \in \mathbb{R}$: real-valued random output variable
- f : function from $\mathbb{R}^p$ to $\mathbb{R}$ (e.g. $f_{\mathbf{w}}$ with parameters $\mathbf{w}$, or a deep neural network)
- $\hat{Y}$ is the prediction for Y : $X \xrightarrow{f(\cdot)} \hat{Y} = f(X)$

How to find the best function f ?

- 01 Measure the difference between $f(X)$ and Y : define a non-negative, real-valued loss $L(Y, f(X))$ that penalizes prediction errors

Common loss functions

SQUARED ERROR

$$L(Y, f(X)) = (Y - f(X))^2$$

0-1 LOSS

$$L(Y, f(X)) = \begin{cases} 1 & \text{if } Y \neq f(X) \\ 0 & \text{if } Y = f(X) \end{cases}$$

CROSS-ENTROPY · CLASSIFICATION

Y one-hot vector, c^* index of the correct class

$$L(Y, f(X)) = -\log(\hat{Y}_{c^*})$$

Expected prediction error: the risk

02 Random variables $\Rightarrow$ assume a joint distribution $P(X, Y)$ exists. The criterion to minimize is the **Expected Prediction Error**, or **Risk**:

$$\begin{aligned} R(f) &= EPE(f) = \mathbb{E}_{P(X,Y)}[L(Y, f(X))] \\ &= \iint L(Y, f(X)) dP(X, Y) \end{aligned}$$

Example — for $L = (Y - f(X))^2$: $R(f) = \iint (y - f(x))^2 p(x, y) dx dy$

(Final) goal: find a hypothesis f^* among a fixed class of functions $\mathcal{F}$ for which the risk is minimal

$$f^* = \underset{f \in \mathcal{F}}{\operatorname{Argmin}} R(f) \quad (1)$$

Cannot solve (1)? The machine learning solution

Problem: $P(X, Y)$ unknown $\Rightarrow R(f)$ cannot be computed

SOLUTION

- Fix $\mathcal{F}$ to a parameterized family $f_{\mathbf{w}}, \mathbf{w} \in \mathbb{R}^d$
- Learn from examples to approximate $R(f_{\mathbf{w}})$

SUPERVISED LEARNING

- Training set $\mathcal{A}_N = \{\mathbf{x}_i, y_i\}_{i=1..N}$: implicit use of $P(X, Y)$ by i.i.d. sampling
- $\mathbf{x}_i \mapsto f_{\mathbf{w}}(\mathbf{x}_i) = \hat{y}_i \longleftrightarrow y_i$

Empirical Risk Minimization (ERM)

Approximate $R(f)$ by averaging the loss function on $\mathcal{A}_N$:

$$R(f) = \iint L(Y, f(X)) dP(X, Y)$$
$$\Rightarrow \text{ERM}(f_{\mathbf{w}}) = \frac{1}{N} \sum_{i=1}^N L(y_i, f_{\mathbf{w}}(\mathbf{x}_i))$$

Requirement: $\{\mathbf{x}_i, y_i\} \sim P(x, y)$ and N large $\Rightarrow$ "good" approximation

NEW OBJECTIVE

$$\mathbf{w}^* = \underset{\mathbf{w} \in \mathbb{R}^d}{\text{Argmin}} \text{ERM}(f_{\mathbf{w}}; \mathcal{A}_N) = \underset{\mathbf{w} \in \mathbb{R}^d}{\text{Argmin}} \frac{1}{N} \sum_{i=1}^N L(y_i, f_{\mathbf{w}}(\mathbf{x}_i))$$

Supervised learning algorithm to solve this **optimization problem**.

Optimization depending on $\mathcal{L}(\mathbf{w})$

- Depending on $\mathcal{L}(\mathbf{w})$, optimization is not always easy
- In this course, $\mathcal{L}(\mathbf{w})$ is (supposed) differentiable for $\mathbf{w} \in \mathbb{R}^d$
- **Definition — gradient:** $\nabla \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{w}} \right) \in \mathbb{R}^d$

Gradient descent algorithm

Initialize $\mathbf{w}^{(0)}$

Repeat $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla \mathcal{L}(\mathbf{w}^{(t)})$

Until $\|\nabla \mathcal{L}(\mathbf{w}^{(t+1)})\|^2 \approx 0$

REMARK · CONVERGENCE

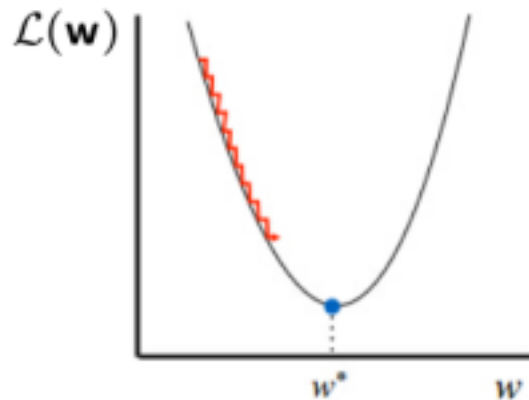
$$\begin{aligned} 0 &\leq \mathcal{L}(\mathbf{w}^{(t+1)}) = \mathcal{L}(\mathbf{w}^{(t)} - \eta \nabla \mathcal{L}(\mathbf{w}^{(t)})) \\ &\approx \mathcal{L}(\mathbf{w}^{(t)}) - \eta \nabla \mathcal{L}(\mathbf{w}^{(t)})^\top \nabla \mathcal{L}(\mathbf{w}^{(t)}) \\ &= \mathcal{L}(\mathbf{w}^{(t)}) - \eta \|\nabla \mathcal{L}(\mathbf{w}^{(t)})\|^2 \\ &\leq \mathcal{L}(\mathbf{w}^{(t)}) \end{aligned}$$

first-order approximation $\Rightarrow$ **local convergence**

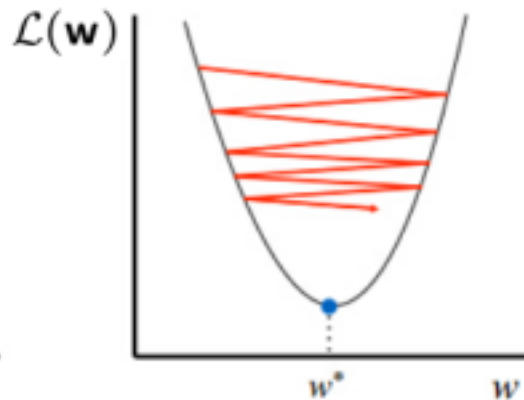
Gradient descent: the learning rate

Update rule: $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{w}}$, η learning rate

Convergence ensured? $\Rightarrow$ provided a "well chosen" learning rate η



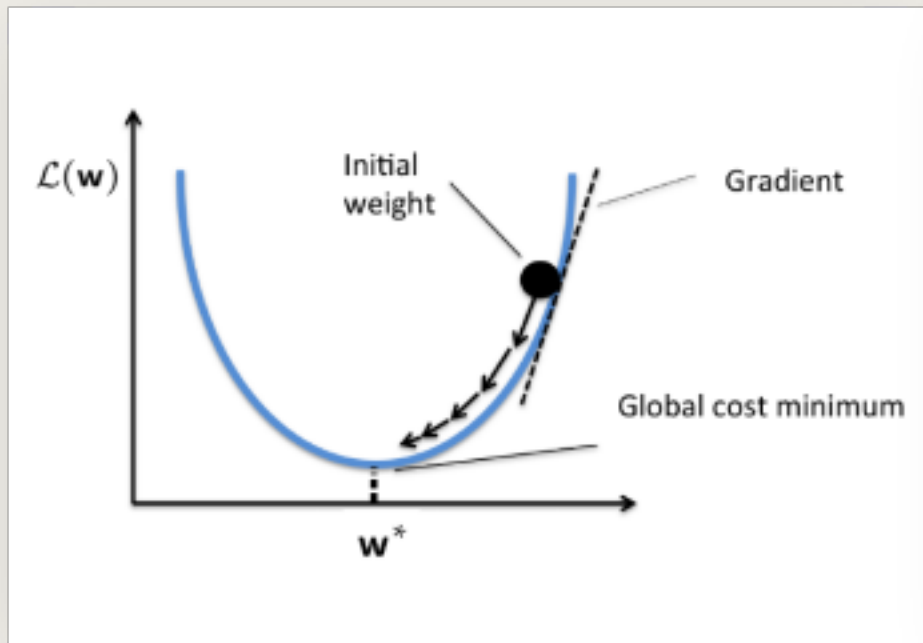
Too small: converge
very slowly



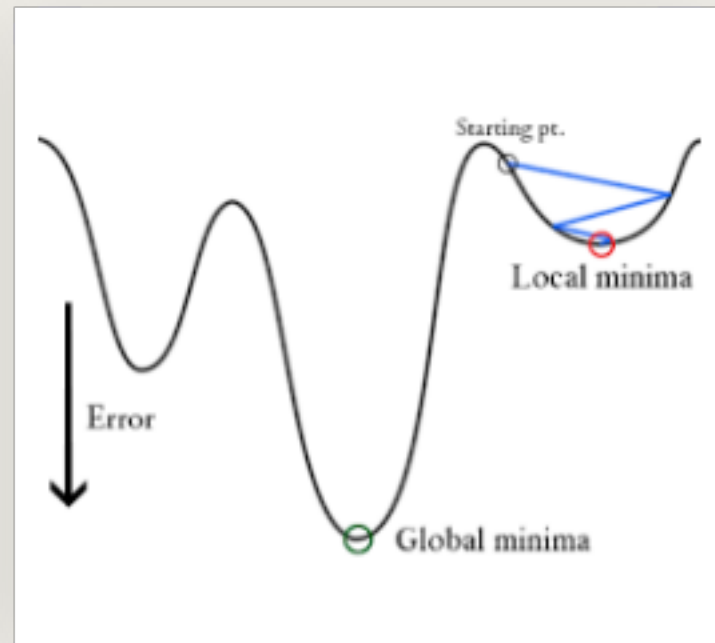
Too big: overshoot and
even diverge

Global minimum?

Convex (a) vs non-convex (b) loss $\mathcal{L}(\mathbf{w})$



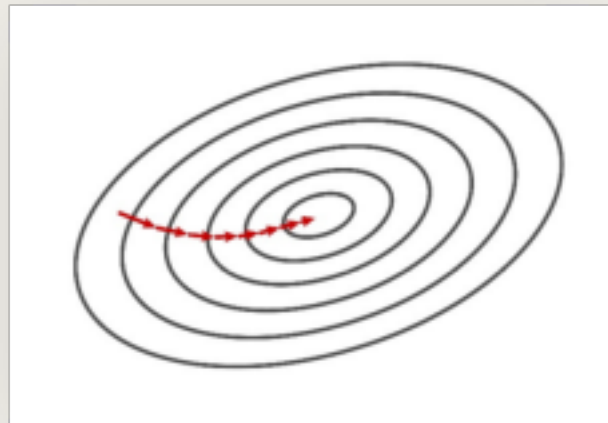
a) Convex function



b) Non-convex function

Neural network training: optimization issues

- Classification loss over the training set (vectorized $\mathbf{w}$, $\mathbf{b}$ ignored): $\mathcal{L}_{CE}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \ell_{CE}(\hat{\mathbf{y}}_i, \mathbf{y}_i^*) = -\frac{1}{N} \sum_{i=1}^N \log(\hat{y}_{c^*,i})$
- Gradient descent: $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \frac{\partial \mathcal{L}_{CE}}{\partial \mathbf{w}}(\mathbf{w}^{(t)}) = \mathbf{w}^{(t)} - \eta \nabla_{\mathbf{w}}^{(t)}$
- Gradient $\nabla_{\mathbf{w}}^{(t)} = \frac{1}{N} \sum_{i=1}^N \frac{\partial \ell_{CE}(\hat{\mathbf{y}}_i, \mathbf{y}_i^*)}{\partial \mathbf{w}}(\mathbf{w}^{(t)})$ scales linearly with:
 - $\mathbf{w}$ dimension
 - training set size
- $\Rightarrow$ **Too slow** even for moderate dimensionality & dataset size



Stochastic Gradient Descent (SGD)

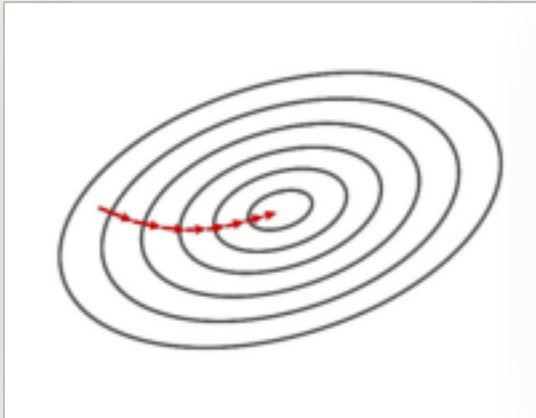
Solution: approximate $\nabla_{\mathbf{w}}^{(t)} = \frac{1}{N} \sum_{i=1}^N \frac{\partial \ell_{CE}(\hat{\mathbf{y}}_i, \mathbf{y}_i^*)}{\partial \mathbf{w}}(\mathbf{w}^{(t)})$ with a subset of examples

SINGLE EXAMPLE · ONLINE

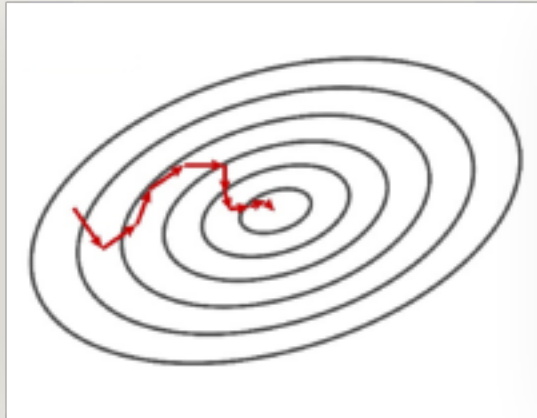
$$\nabla_{\mathbf{w}}^{(t)} \approx \frac{\partial \ell_{CE}(\hat{\mathbf{y}}_i, \mathbf{y}_i^*)}{\partial \mathbf{w}}(\mathbf{w}^{(t)})$$

MINI-BATCH · $B < N$ EXAMPLES

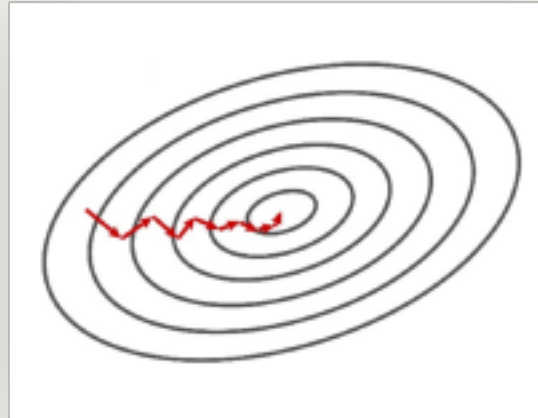
$$\nabla_{\mathbf{w}}^{(t)} \approx \frac{1}{B} \sum_{i=1}^B \frac{\partial \ell_{CE}(\hat{\mathbf{y}}_i, \mathbf{y}_i^*)}{\partial \mathbf{w}}(\mathbf{w}^{(t)})$$



Full gradient



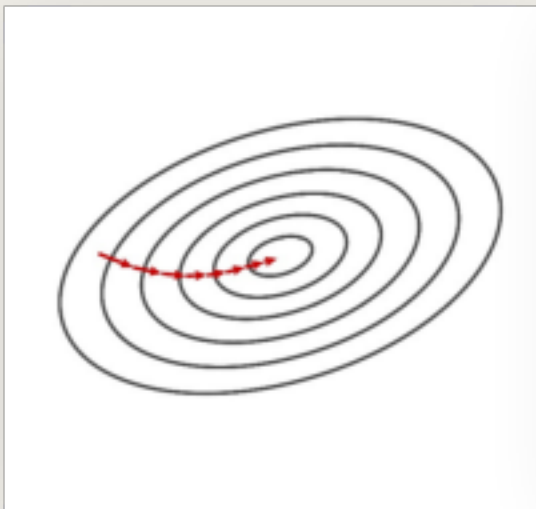
SGD (online)



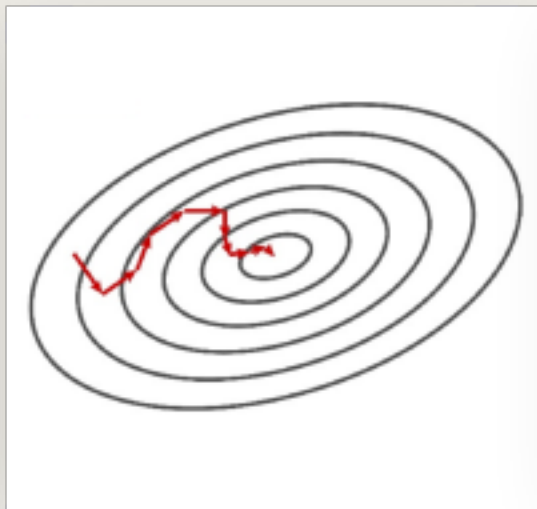
SGD (mini-batch)

SGD: an approximation of the true gradient

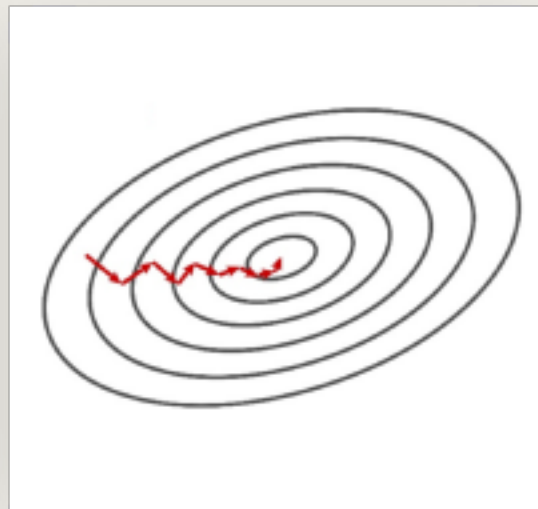
- Noisy gradient can lead to a bad direction and increase the loss
- **But:** many more parameter updates — online $\times N$, mini-batch $\times \frac{N}{B}$
- **Faster convergence:** at the core of deep learning for large-scale datasets



Full gradient



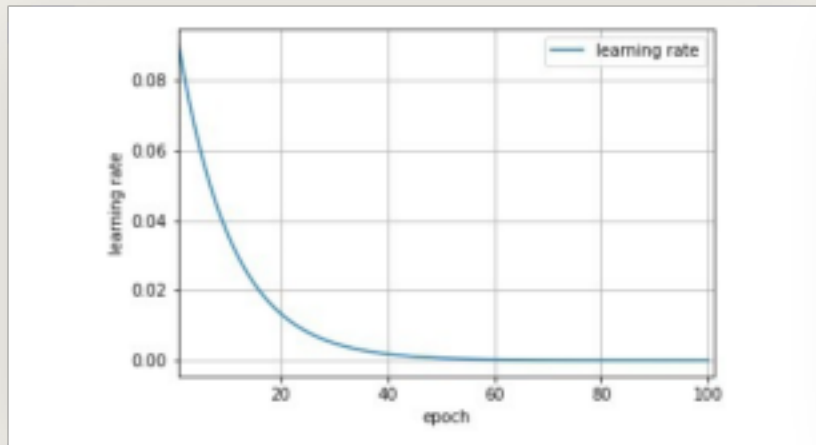
SGD (online)



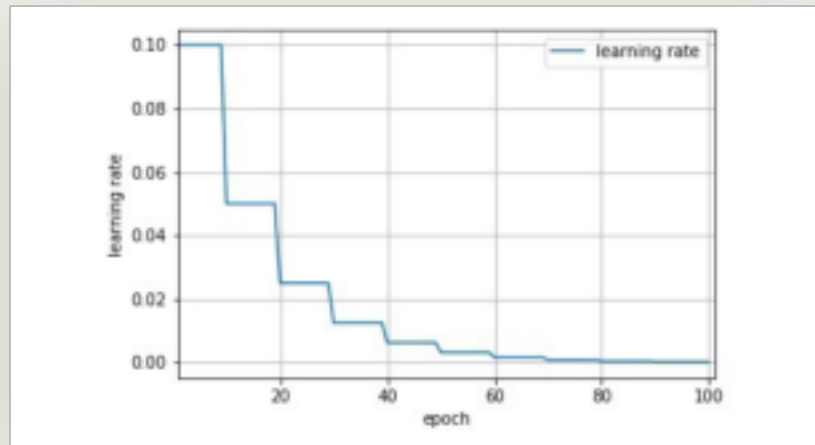
SGD (mini-batch)

Learning rate decay

- Gradient descent: $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla_{\mathbf{w}}^{(t)}$ — how to set η ? $\Rightarrow$ open question
- **Learning rate decay:** decrease η as training progresses
 - Inverse (time-based): $\eta_t = \frac{\eta_0}{1+r \cdot t}$, r decay rate
 - Exponential: $\eta_t = \eta_0 \cdot e^{-\lambda t}$
 - Step: $\eta_t = \eta_0 \cdot r^{\lfloor t/t_u \rfloor}$, ...



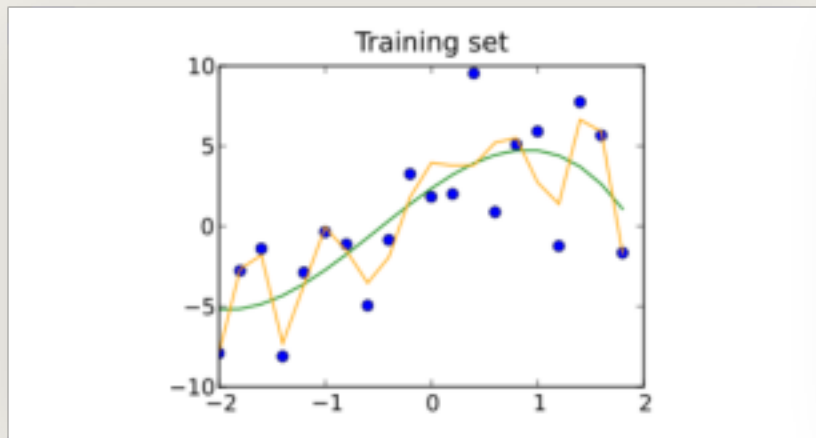
Exponential decay ($\eta_0 = 0.1$, $\lambda = 0.1$)



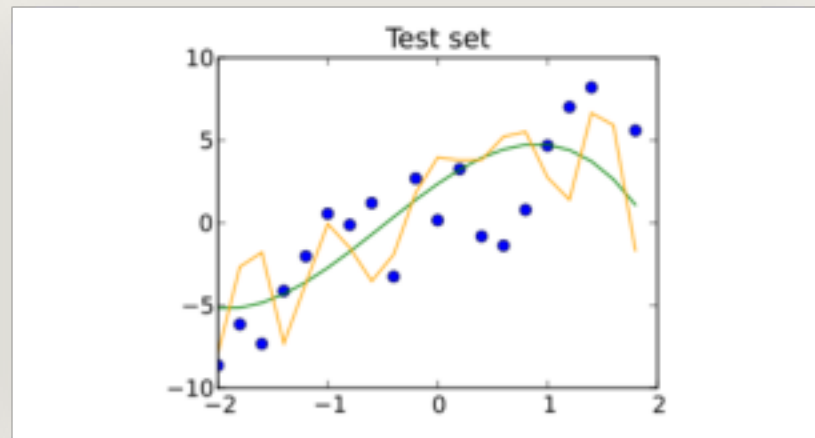
Step decay ($\eta_0 = 0.1$, $r = 0.5$, $t_u = 10$)

Generalization and overfitting

- Learning: minimizing the classification loss $\mathcal{L}_{CE}$ over the training set
- Training set: a sample representing the data vs labels distribution
- Ultimate goal: low prediction error on the **true (unknown) data distribution**



$L_{\text{train}} = 4 \cdot L_{\text{test}} = 15$

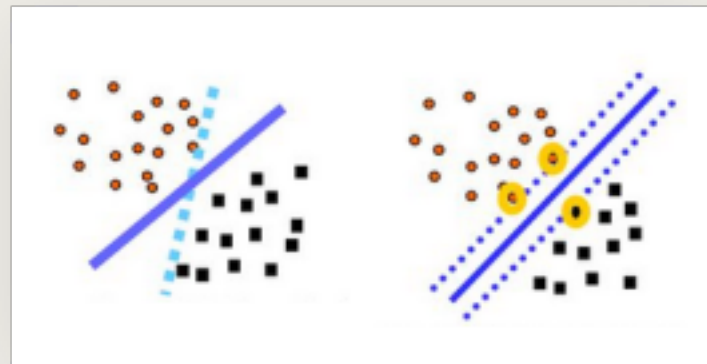


$L_{\text{train}} = 9 \cdot L_{\text{test}} = 13$

⇒ Optimization $\neq$ machine learning ⇒ Generalization / overfitting

Regularization

- **Regularization:** improving generalization, i.e. test ($\neq$ train) performance
- Structural regularization: add a **prior** $R(\mathbf{w})$ to the training objective: $\mathcal{L}(\mathbf{w}) = \mathcal{L}_{CE}(\mathbf{w}) + \alpha R(\mathbf{w})$
- L^2 regularization — **weight decay**: $R(\mathbf{w}) = \|\mathbf{w}\|^2$
 - Commonly used in neural networks
 - Theoretical justifications, generalization bounds (SVM)
- Other possible $R(\mathbf{w})$: L^1 regularization, dropout, etc.



L^2 regularization: interpretation

"Smooth" interpretation of L^2 regularization — Cauchy-Schwarz:

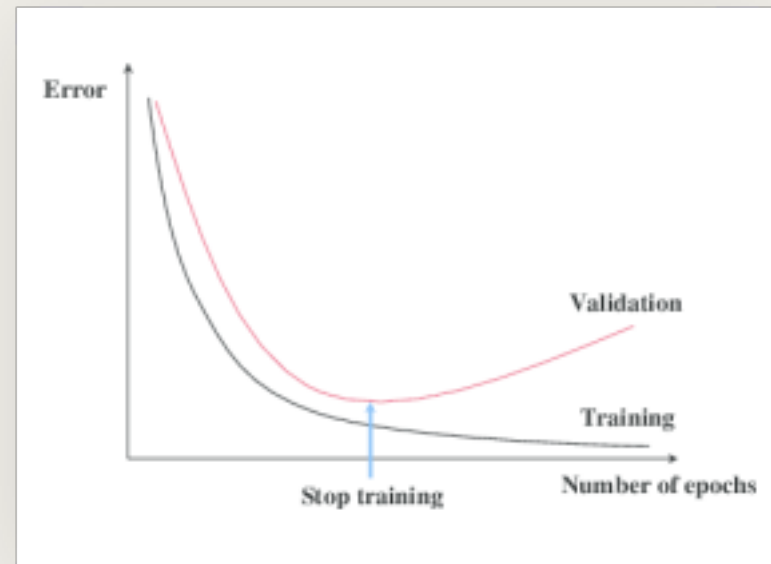
$$(\langle \mathbf{w}, (\mathbf{x} - \mathbf{x}') \rangle)^2 \leq \|\mathbf{w}\|^2 \|\mathbf{x} - \mathbf{x}'\|^2$$

- Controlling $\|\mathbf{w}\|^2$: "small" variation between inputs $\mathbf{x}$ and $\mathbf{x}' \Rightarrow$ small variation in the neuron prediction $\langle \mathbf{w}, \mathbf{x} \rangle$ and $\langle \mathbf{w}, \mathbf{x}' \rangle$

$\Rightarrow$ Supports **simple**, i.e. smoothly varying, prediction models

Regularization and hyper-parameters

- **Neural networks:** hyper-parameters to tune
 - **Training:** learning rate, weight decay, learning rate decay, # epochs, etc.
 - **Architecture:** number of layers, number of neurons, non-linearity type, etc.
- **Hyper-parameter tuning** $\Rightarrow$ improve generalization: estimate performance on a **validation set**



NEXT · LECTURE 3

Neural nets for image classification

Clement Rambour

`clement.rambour@sorbonne-universite.fr`

`clementrambour.github.io/teaching-rdfia`